MLDS 490 Advanced Algorithms for Artificial Intelligence

Assignment #2

Yuxuan Liang

GitHub username: `yuxuanliang1`

Repository name: `MLDS490-Assignment2`

Repository URL: https://github.com/yuxuanliang1/MLDS490-Assignment2

November 11, 2026

Abstract

This report evaluates original PyTorch implementations of Classic Deep Q-Networks in CartPole-v1 and MsPacman-v5. Both agents use randomized experience replay, independent target networks, Huber loss, Adam, and gradient clipping. CartPole uses a two-hidden-layer multilayer perceptron and discount factor 0.95; MsPacman uses a single-frame convolutional network and discount factor 0.99. All neural network training ran sequentially on one RTX 5060 Ti on Northwestern's deepdish5. The selected CartPole policy obtained a mean undiscounted return of 499.266 with population standard deviation 11.73 over exactly 500 greedy episodes; 498 episodes reached the 500-step limit. MsPacman completed 3,114 training games before its decision budget expired and obtained a mean score of 1,501.06 with population standard deviation 493.72 over a separate 500-game evaluation. Controlled target-network, replay, discount, exploration, and seed experiments expose substantial sensitivity, including finite Q-value inflation. Independent Bellman diagnostics, altered initial states, and checkpoint-recovery experiments test specific correctness and reliability claims. Action maps and image-conditioned Q preferences complement reward curves. These measurements support learning, while limited full-budget replication, reused validation panels, and remaining failures preclude claims of universal convergence or robustness.

1 Introduction

Reinforcement learning estimates the consequences of actions from interaction. Deep Q-Networks (DQN) approximate action values with a neural network, select actions from those estimates, and improve them using bootstrapped targets [4]. Experience replay reuses transitions in randomly sampled batches; a separate target network slows changes in the regression targets. These are the stabilization mechanisms emphasized in the Week 6 lecture and lab [2, 5].

The two assignment environments exercise different aspects of this method. CartPole-v1 exposes cart position, cart velocity, pole angle, and angular velocity. Two actions apply horizontal force, and the goal is to balance for all 500 steps. MsPacman-v5 instead supplies RGB images and nine discrete commands. Visual compression, hidden motion, irregular rewards, and longer credit assignment make its value estimation and exploration substantially harder. Its raw game score and CartPole's unit-reward return are different quantities and are not compared numerically as a common measure of difficulty.

The objectives are to implement inspectable Classic DQN, measure learned greedy policies on exactly 500 episodes per game, and test whether the algorithm behaves as expected under controlled interventions. The assignment's required discount factors, 0.95 and 0.99, remain the primary settings. Validation includes negative and inconclusive results rather than treating passing software tests as evidence of successful learning.

2 DQN methodology

For a policy π, the action-value function is

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a\right]. \tag{1}$$

For a Markov state, the optimal value satisfies the Bellman relation

$$Q^{*}(s,a) = \mathbb{E}\left[r + \gamma(1-d)\max_{a'} Q^{*}(s',a') \mid s,a\right], \tag{2}$$

where d indicates genuine termination. In the visual experiment a single image is used as a state representation; it need not be Markov because motion is hidden. This is a representational limitation of applying the same update there.

The online network Q_θ outputs one real-valued estimate per action. Its outputs estimate discounted returns, with no softmax or probability normalization. Training chooses a uniformly random action with probability ϵ and otherwise $\arg\max_a Q_\theta(s,a)$. With n global interaction decisions,

$$\epsilon(n) = \epsilon_0 + (\epsilon_f - \epsilon_0)\min(n/N_\epsilon, 1). \tag{3}$$

Evaluation fixes $\epsilon = 0$; environment stochasticity, including Atari sticky actions, remains active.

The replay ring stores $(s, a, r, s', \texttt{terminated}, \texttt{truncated})$ in preallocated NumPy arrays. It overwrites the oldest data at capacity. Baseline minibatches are uniformly sampled without replacement within each batch and resampled at each update. This reduces temporal correlation but does not make successive overlapping batches statistically independent. Updates start only after the configured warmup.

The target network Q_{θ^-} starts as a copy of the online network, has gradients disabled, and supplies the detached target

$$y_i = r_i + \gamma(1-d_i)\max_{a'} Q_{\theta^-}(s_i', a'). \tag{4}$$

This is Classic DQN: action maximization and evaluation both use the target network, rather than the Double DQN alternative. With residual $\delta_i = Q_\theta(s_i,a_i) - y_i$, the minimized minibatch loss is

$$L(\theta) = \frac{1}{B}\sum_{i=1}^{B} h(\delta_i), \qquad h(\delta) = \begin{cases} \frac{1}{2}\delta^2, & |\delta| \le 1, \\ |\delta| - \frac{1}{2}, & |\delta| > 1. \end{cases} \tag{5}$$

Adam updates only online parameters; the gradient vector is clipped to norm 10. Logged gradient norms are measured *before* clipping and can exceed 10. Hard synchronization copies online weights after each specified number of optimizer updates. Between copies, lagged targets limit the feedback from changing online predictions into their own regression labels. This reduces one source of instability, without guaranteeing convergence under nonlinear approximation. NaN/Inf checks cover Q-values, targets, losses, gradients, and updated weights. Finite but grossly inflated estimates still require scientific diagnostics.

2.1 Termination, truncation, and reward accounting

Gymnasium separates genuine MDP termination from external time limits [7]. Interaction stops on either flag; no further step is issued until reset. Both flags are retained, and the final transition's reward is counted. Only `terminated` masks Equation (4). For example, with $r = 1$, $\gamma = 0.95$, and next maximum Q-value 4, the target is 4.8 for an ordinary or time-limited transition, but 1 for genuine failure. Masking `terminated OR truncated` would incorrectly model a time cutoff as an absorbing state. If both flags occur, termination masks the continuation. This is a continuing-task bootstrap convention; it does not encode a separate finite-horizon value function with remaining time in the observation.

2.2 Training and model selection protocol

Each iteration executes an epsilon-greedy action, records the complete transition, inserts it into replay, optionally samples a minibatch and applies Equations (4)–(5), and then synchronizes the target when due. Episode summaries include return, length, epsilon, loss, gradient norm, Q statistics, decisions, updates, and elapsed time. Separate environments provide periodic greedy validation. A strictly improved validation mean saves an immutable best checkpoint; earliest checkpoints win ties. Final snapshots are also retained. Episode-boundary budget checks permit small overshoots. Neither experiment used a convergence-based stopping rule.

Training rewards describe exploratory behavior under changing weights; validation means select snapshots; the 500-episode panels assess frozen selected policies. All reported within-model standard deviations use the population convention $\mathrm{SD}_0 = \sqrt{\sum_i (R_i - \bar{R})^2 / n}$. Between-training-run variation is identified separately and is not the standard deviation of individual evaluation episodes.

3 CartPole-v1

3.1 Implementation and training configuration

The state $(x, \dot{x}, \theta, \dot{\theta})$ is used directly as float32. The MLP has architecture $4 \to 128 \to 128 \to 2$, with ReLU after each hidden layer and linear action-value outputs. Action 0 pushes left and action 1 pushes right. Table 1 reports the selected run's saved configuration rather than assuming that current CLI defaults reproduce its budget.

Seed 33 completed 942 episodes, 80,014 decisions, 79,015 optimizer updates, and 158 target copies in 180.1 seconds. Its best validation mean was 500 at episode 942, selecting `best_ep0942.pt`. Validation uses reset seeds 50,000–50,009. Among baseline training seeds 11, 22, and 33, this model had the highest validation mean. The smaller seed-11 and seed-22 validation maxima were 407.6 and 473.7. All three seeds' test results are disclosed in Section 5.1.

3.2 Learning curves and the Q-value scale

For episode e, the Q statistic is $\max_a Q_{\theta_e^{\mathrm{end}}}(s_{e,0}, a)$: the episode's initial observation is evaluated using the online weights *after* that episode's updates. Figure 1 therefore mixes changing weights with independently changing reset states; it is not a maximum over the trajectory or replay buffer.

Table 1: CartPole primary configuration (training seed 33).

Setting	Value
Environment; discount	CartPole-v1; $\gamma = 0.95$
Optimizer; learning rate	Adam; 5×10^{-4}
Replay capacity; warmup	50,000; 1,000 transitions
Batch size; update frequency	128; every decision after warmup
Target synchronization	Hard copy every 500 optimizer updates
Loss; gradient norm cap	Huber ($\beta = 1$); 10
Exploration	$1.0 \rightarrow 0.05$ linearly over 20,000 decisions
Validation	Every 25 episodes and final episode; 10 greedy resets
Limits	1,000 episodes or 80,000 decisions; native 500-step episodes
Stopping	Budget at episode boundary; early stopping disabled

Its observed range is 0.039–20.81, with final value 20.67. Continuing unit rewards have scale

$$\sum_{t=0}^{\infty} \gamma^t = \frac{1}{1-\gamma}, \qquad 0 \le \gamma < 1, \tag{6}$$

which equals 20 at $\gamma = 0.95$; the 500-step discounted sum is effectively 20 as well. Thus undiscounted reward 500 does not imply Q-values near 500. Slight overshoot of 20 is possible under approximation; the network does not enforce this theoretical reference.

The early staircase also reveals the timing of value propagation. Replay warmup ends at decision 1,000 in episode 45. Matching update logs to episode boundaries places the first hard copies at decisions 1,499, 1,999, 2,499, and 2,999, within episodes 67, 90, 106, and 123. Across those episodes, the plotted Q statistic rises from 1.051 to 1.970, 2.009 to 2.699, 2.909 to 3.827, and 3.773 to 4.492, respectively. These rises align with copy-containing episodes, consistent with lagged Bellman targets producing successive value plateaus. A copy changes the target network, not the online prediction directly; subsequent regression updates can propagate the new target level. Episode-end sampling cannot isolate that mechanism causally, and reset states and episode lengths also vary. The spacing in episodes is therefore irregular rather than a fixed 20-episode schedule.

Figure 2 shows all exploratory returns and the strict trailing 100-episode mean, first defined at episode 100. The first 100 episodes average 22.83; the final 100 (episodes 843–942) average 177.32. Epsilon first reaches its logged floor of 0.05 in episode 496, ending at 20,059 decisions, but the first 500-step training episode occurs only at episode 870; six training episodes reach 500 in total. Greedy validation then rises from 162.8 at episode 900 to 492.8 at 925 and 500 at 942. Thus the trailing mean combines earlier weaker policies with exploratory actions, whereas the selected frozen policy uses epsilon zero and achieves 499.266 in the independent evaluation. The difference reflects both the training timeline and action-selection protocol. Improvement so near the budget boundary leaves little evidence about stability under continued training; longer runs and independent-seed replication are priorities in Section 6.2.

3.3 Exactly 500 greedy evaluation episodes

The selected checkpoint was evaluated with epsilon zero on distinct reset seeds 100,000–100,499. Table 2 and Figure 3 report the complete panel. Exactly 498/500 episodes reached 500 steps; the

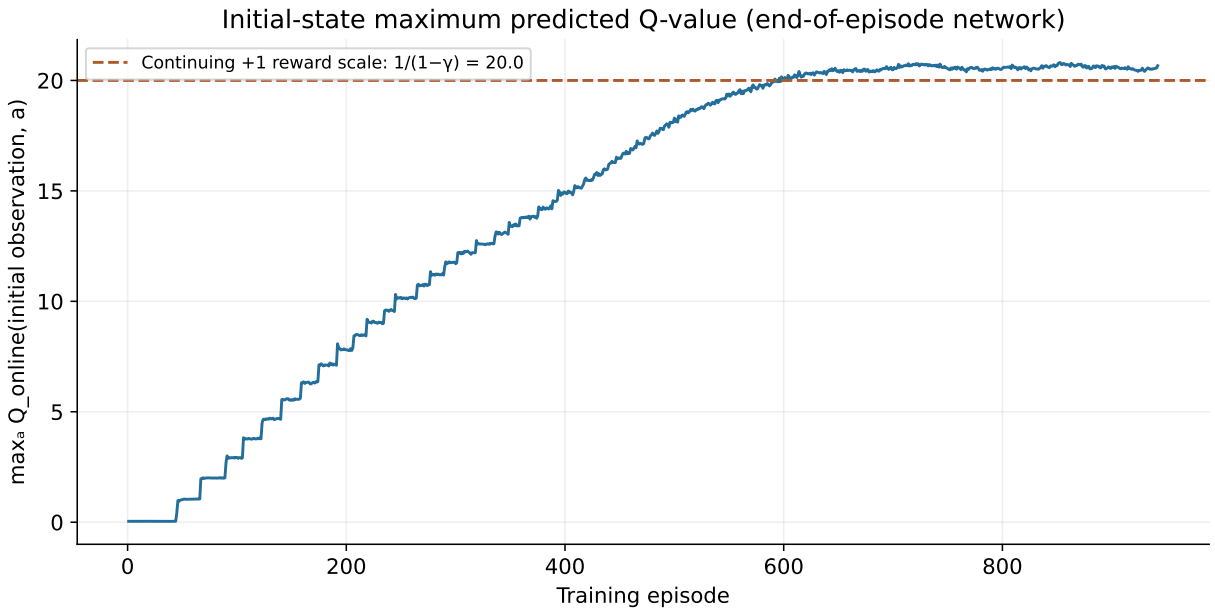


Figure 1: CartPole maximum action value at each episode's initial observation, using the online network after that episode's updates. The discounted unit-reward reference is 20, not the undiscounted episode limit of 500.

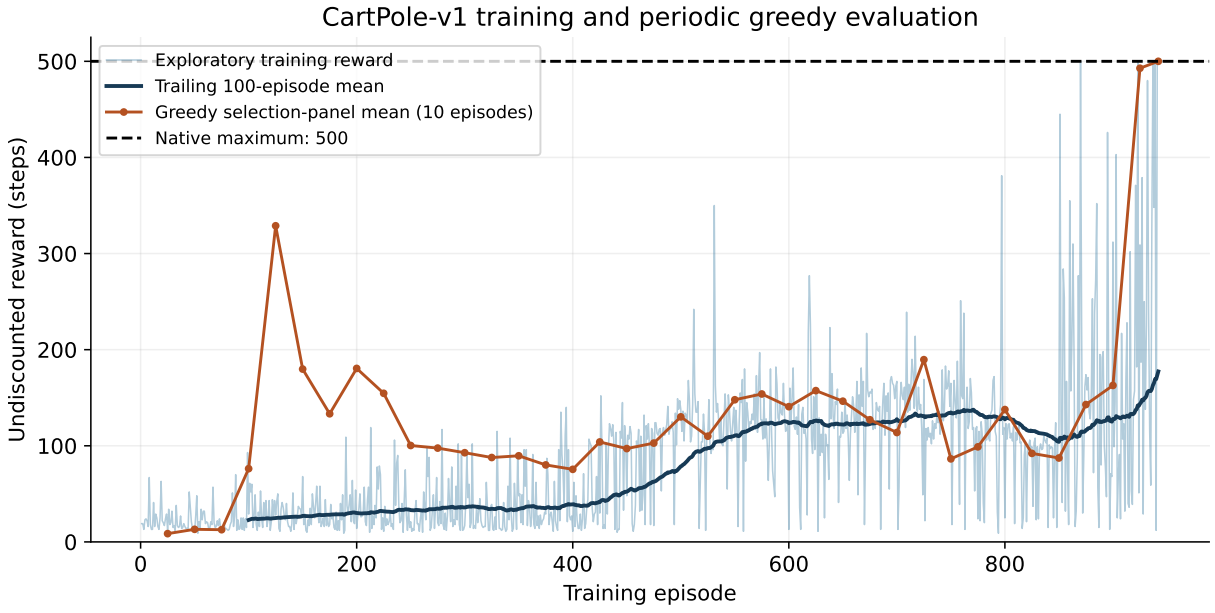


Figure 2: CartPole seed-33 exploratory training returns, trailing 100-episode mean, and separate greedy validation means. The reward-500 line marks the native time limit. Validation episodes are excluded from the moving average.

remaining returns were 346 and 287 at seeds 100,211 and 100,261. The high mean therefore coexists with two genuine failures. A different final-snapshot diagnostic used 100 other seeds and is not substituted for this evaluation. The saved evaluation rows contain only reset seed, reward, length, and end flags; they contain neither final observations nor threshold-specific termination reasons. Thus the two failures cannot be attributed to cart displacement or pole angle from these records. They establish limited robustness, without identifying its physical mechanism.

Table 2: Frozen selected policies: exactly 500 greedy episodes per environment. Standard deviations describe individual returns within each model, with $ddof = 0$. The games' reward units differ.

Statistic	CartPole (unit rewards)	MsPacman (raw score)
Training seed; selected episode	33; 942	41; 3,114
Mean	499.266	1,501.06
Population standard deviation	11.73	493.72
Median	500	1,430
Minimum / maximum	287 / 500	590 / 4,800
True terminations / truncations	2 / 498	500 / 0
Perfect 500-step episodes	498/500 (99.6%)	Not applicable

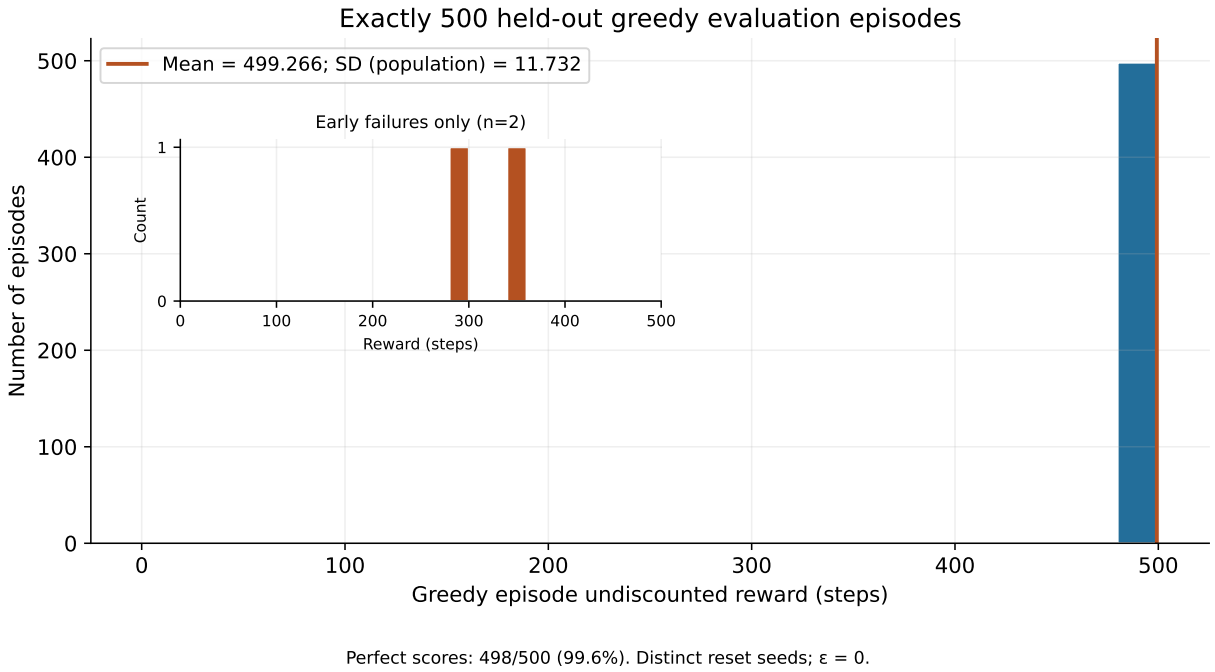


Figure 3: CartPole selected-checkpoint reward histogram for exactly 500 greedy episodes. Mean 499.266, population SD 11.73, and 498 perfect episodes. The two low-return episodes are retained in all statistics.

3.4 Non-standard visualization: a control-policy slice

Figure 4 evaluates both action values on a 161×161 grid of pole angle and angular velocity, with $x = \dot{x} = 0$. Decision regions use the argmax, and contours show $Q(s, \text{right}) - Q(s, \text{left})$; the zero

contour marks equal preferences. The saved grid is `metrics/policy_grid.csv` within the seed-33 run.

The map is asymmetric. Near $\theta = \pm 0.05$ rad and zero angular velocity, both sampled states select right; the probes at zero angle and angular velocities ± 0.5 rad/s also select right. These do not follow a symmetric local corrective rule. Larger paired probes near $(-0.10, -1)$ and $(0.10, 1)$ select left and right, respectively, consistent with moving the cart beneath the falling pole. These observations do not establish optimality: cart position and velocity are suppressed, and closed-loop trajectories determine actual stability. Agreement with the heuristic "push right if $\theta + 0.2\dot{\theta} > 0$" is only 49.97% on the *uniformly weighted central subgrid*, $|\theta| \leq 0.05$ rad and $|\dot{\theta}| \leq 0.5$ rad/s. It is not an on-policy agreement rate or a measured probability of correct control; the heuristic is not a correctness oracle.

The full displayed grid spans $\theta \in [-0.2094, 0.2094]$ rad and $\dot{\theta} \in [-2, 2]$ rad/s, much wider than the standard reset range of $[-0.05, 0.05]$ for each state component. The 500 evaluation records have no state trajectories, while the training transitions are exploratory and use changing weights. The available ten-step greedy trace (Table 6) is too short to estimate typical occupancy or locate rarely visited disagreement regions. It also reaches $\dot{x} = 1.131$ m/s, outside the map's fixed $\dot{x} = 0$ slice. Consequently, 498/500 successful closed-loop episodes are compatible with poor heuristic agreement on this slice: the two measurements weight different states and suppress different information. Neither result establishes sensible control throughout the displayed grid. The altered-state test in Section 5.1 provides a direct, more demanding robustness check.

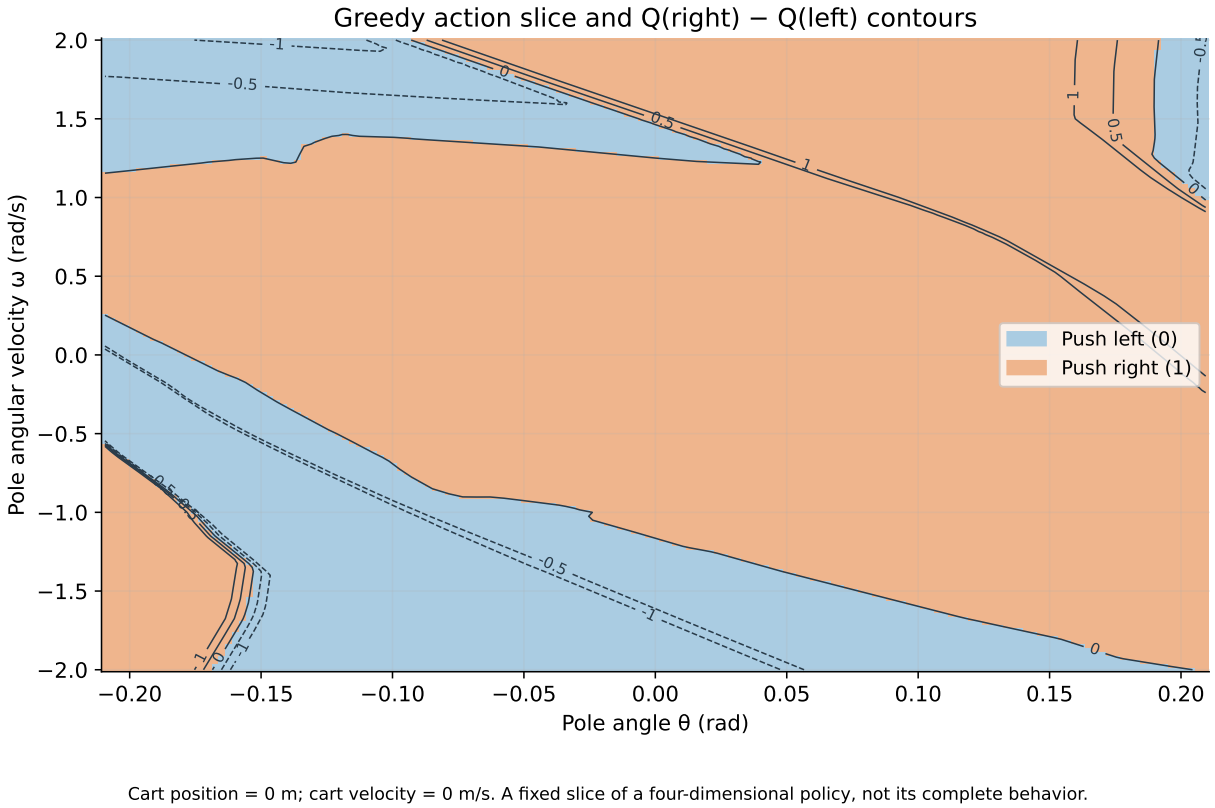


Figure 4: Greedy CartPole action regions and action-value difference contours on the fixed slice $x = \dot{x} = 0$. Angle is in radians; angular velocity is in rad/s. This is a two-dimensional slice of a four-dimensional policy, not a complete recovery-region map.

3.5 CartPole: edge cases and validation

Early failure and the native 500-step time limit require different Bellman treatment: both end interaction and retain the final reward, but only genuine failure masks continuation. A one-step failure returns 1; with next maximum Q 4, the known target is 1 for termination versus 4.8 for truncation (Section 5.3). The selected policy reaches 500 steps in 498/500 standard resets (99.6%), compared with 293/405 altered-state episodes (72.35%) under unchanged dynamics. High standard-reset success and recovery from larger state perturbations are therefore distinct properties. The controlled target/replay/discount/seed and robustness experiments in Section 5.1, together with Table 7, quantify these claims rather than relying on input-format tests.

4 MsPacman-v5

4.1 Environment, representation, and architecture

The installed ALE registers `ALE/MsPacman-v5`. The implementation creates the assignment's literal `MsPacman-v5` ID as an alias with identical entry point and configuration, rejecting conflicting registrations. A recorded test compares RGB observations, rewards, and end flags for 120 common actions under the alias and canonical environment. This changes the registration name, not the game dynamics. ALE is the emulator interface [1]; explicit evaluation protocols matter, particularly sticky actions [3].

The environment supplies $210 \times 160 \times 3$ uint8 RGB images and nine commands: NOOP, UP, RIGHT, LEFT, DOWN, UPRIGHT, UPLEFT, DOWNRIGHT, DOWNLEFT (indices 0–8). Native frameskip is four, sticky-action probability is 0.25, and the native limit is 108,000 raw frames per game. There is no additional action-repeat wrapper, random no-op reset, frame stack, reward clipping, or episodic-life termination. Life loss continues the same game. Headless execution does not request a display. Discounting applies per agent decision, not per raw emulator frame.

Preprocessing follows the assignment exactly. The slice `obs[1:176:2, ::2]` crops and downsamples to 88×80 pixels. RGB channels are summed in signed int32 to prevent overflow. Every sum equal to $210 + 164 + 74 = 448$ is set to zero; this is an intensity-sum mask, not a semantic background detector. Integer division by three followed by subtraction of 128 produces int8 intensities. The HWC array is transposed to shape $(1, 88, 80)$, and the CNN converts it to float32 and divides by 128. The representable input range is $[-1, 127/128]$. This differs from a conventional stacked-frame Atari pipeline.

The CNN uses three unpadded convolutions: $1 \to 32$ channels with kernel 8 and stride 4; $32 \to 64$ with kernel 4 and stride 2; and $64 \to 64$ with kernel 3 and stride 1. ReLU follows each. The resulting $64 \times 7 \times 6 = 2,688$ features feed a 512-unit ReLU layer and nine linear Q outputs. There are 1,453,225 parameters. Replay stores current and next int8 images separately. Its preallocated arrays occupy 704,700,000 bytes (0.656 GiB), avoiding float32 image storage; a RAM guard rejects oversized allocation requests. Table 3 summarizes the primary training configuration.

4.2 Training exposure and learning evidence

Training completed 3,114 games, 2,000,528 agent decisions, and 7,992,770 measured raw frames. It performed 497,633 optimizer updates and 99 hard target copies. Elapsed training time was 3,268.5 seconds (54.48 minutes), including periodic validation and recovery writes. Peak allocated CUDA

Table 3: Saved MsPacman primary configuration, seed 41. A decision is an agent action, advancing up to four raw frames; optimizer cadence is a separate quantity.

Setting	Value
Environment; discount	MsPacman-v5; $\gamma = 0.99$
Optimizer; learning rate	Adam; 10^{-4}
Replay capacity; warmup	50,000; 10,000 decisions
Batch size; update frequency	32; every four decisions after warmup
Target synchronization	Hard copy every 5,000 optimizer updates
Loss; gradient norm cap	Huber ($\beta = 1$); 10
Exploration	$1.0 \rightarrow 0.1$ linearly over 500,000 decisions
Validation	Every 100 games and at end; 10 fixed greedy resets
Planned limits	4,000 games, 2,000,000 decisions, or 4 hours
Recovery cadence	Every 250 completed games and at final boundary

tensors occupied 103.04 MiB; final process RSS was 3.31 GiB. CUDA context and reserved memory are additional to allocated tensor memory. The decision cap was checked at a game boundary and overshot by 528 decisions. Training did not complete the assignment's suggested 4,000–6,000-game scale and did not stop on a convergence claim.

Figure 5 uses a fixed panel of 32 unique images collected before training by random play at seed 410,000, every 25 decisions. After each training game it plots the *mean over images of the maximum over actions*. The band is population SD across these fixed images, not a confidence interval over runs. The mean statistic ranged from 0.026 to 85.20 and finished at 84.01. Its increase shows changing value estimates; it does not establish calibration. Raw rewards are not constant unit rewards, so $1/(1 - 0.99) = 100$ is not a bound on MsPacman Q-values or its undiscounted game score.

Figure 6 supplies direct performance evidence. First-100 and last-100 exploratory means were 272.1 and 994.2. Greedy validation uses seeds 600,000–600,009: the untrained model scored 210 on average, the episode-100 model 110, the episode-1,600 model 707, and the selected episode-3,114 model 1,387. Validation fluctuates, including means 1,205 at episode 2,800, 811 at 2,900, and 830 at 3,000. Uniform random actions on that panel scored 252 (population SD 77.56).

Epsilon reaches its floor of 0.10 during episode 888, which ends at 500,500 decisions. Afterward, arithmetic means of individual training scores increase from 806.7 in episodes 1,001–1,500 to 905.4 in 2,501–3,000, despite an intervening decline to 782.2 in 1,501–2,000. Means of the saved greedy validation checkpoint scores rise from 751.3 over episodes 1,100–2,000 (10 checkpoints) to 966.0 over 2,100–3,114 (12, including the final checkpoint). These later gains cannot be explained solely by declining random-action frequency: epsilon is constant, although 10% exploration remains. They support continued policy learning with substantial fluctuations; the checkpoint averages reuse the same reset panel.

Between episodes 1,000 and 3,114, the fixed-reference Q mean rises from 19.19 to 84.01 (4.38-fold; Figure 5), while the corresponding trailing-100 training score rises from 783.7 to 994.2 (1.27-fold; Figure 6). Discounted predictions on early reference images and undiscounted exploratory whole-game means need not grow proportionally; this comparison does not establish overestimation or maximization bias, which the proposed Double DQN and return-calibration experiments would investigate (Section 6.2).

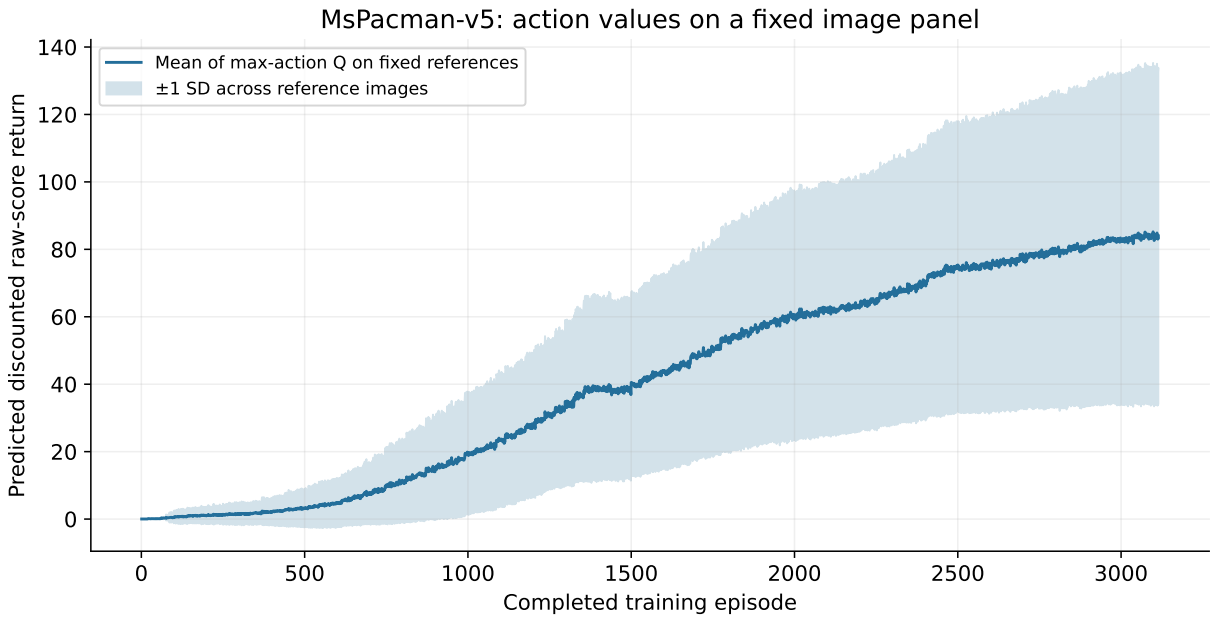


Figure 5: MsPacman maximum Q-values on a fixed 32-image panel, averaged over images after each completed training game. Shading is SD across images, not between training seeds. Images come from early random play.

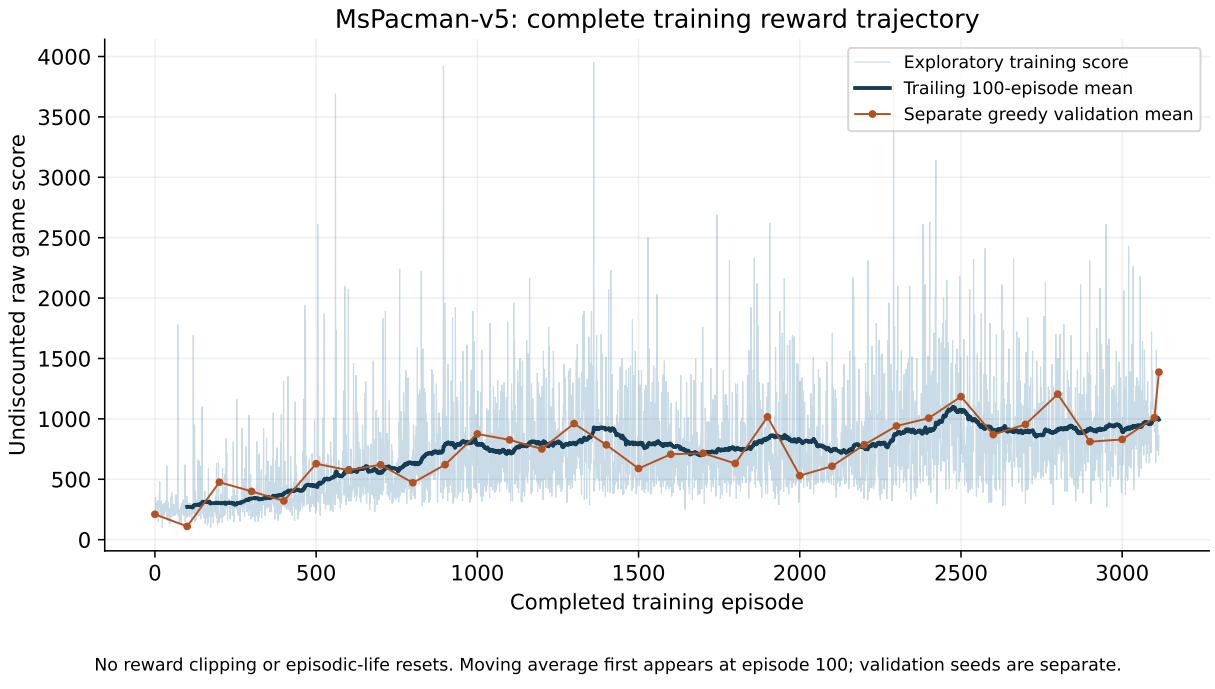


Figure 6: All MsPacman exploratory scores, strict trailing 100-game mean, and separate greedy validation means. Rewards are raw and include every life. Performance regressions qualify the overall upward learning trend.

4.3 Selected policy and 500-game evaluation

The highest ten-seed validation mean selected `best_ep03114.pt` at the last completed game; the final model was separately saved. Final evaluation used epsilon zero with seeds 900,000–900,499, disjoint from selection and short-run diagnostics. Exactly 500 games completed in 251.7 seconds. Mean score was 1,501.06, population SD 493.72, median 1,430, and range 590–4,800 (Table 2; Figure 7). All ended by genuine game-over, with no frame-limit truncations. Mean length was 856.52 decisions or 3,423.08 raw frames. Native repeats can end early on game-over, so measured raw frames need not equal exactly four times decisions. The spread shows considerable conditional performance variation across resets; these games do not measure variability between independently trained full-budget models.

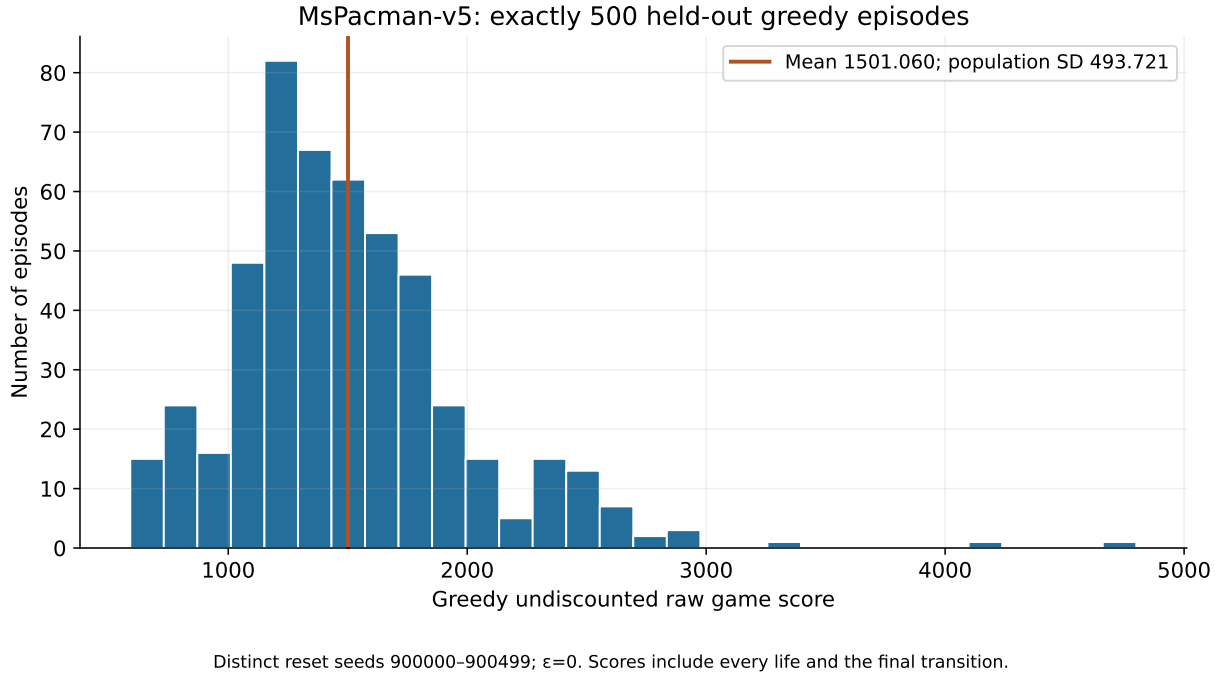


Figure 7: Exactly 500 held-out MsPacman games from the selected greedy checkpoint. Mean 1,501.06 and population SD 493.72. Sticky actions remain enabled; all scores include the final transition and all lives.

4.4 Non-standard visualization: image-conditioned action preferences

Figure 8 displays six of the fixed reference images and all nine saved action values. Across the complete 32-image panel, seven different commands are greedy, 93.75% of greedy commands differ from initialization, and the median top-two Q margin is 0.291. In the displayed examples, reference 0 prefers NOOP, 6 prefers UP, 18 prefers UPLEFT, and 31 prefers LEFT. Preferences therefore vary with the observed image; a constant-action policy does not explain this panel. Small margins and differing absolute scales also show why a Q margin is neither a calibrated confidence nor a probability of an optimal action.

Reference 0 is specifically the reset observation at seed 410,000, saved before the first random action; subsequent references are sampled every 25 decisions. Its RGB image shows the opening maze with score zero. Its maximum Q is 4.73, compared with 104.09–127.04 for the other five displayed

images. Reapplying the specified preprocessing to all six RGB frames exactly reproduces their saved int8 inputs. Reference 0 spans $[-128, 27]$, like four of the other five examples, and its mean intensity is -41.05 versus -41.09 for reference 6. This rules out a mismatch between the saved RGB frame and processed input, but not a sensitivity to local pixel patterns. The collection time establishes an opening observation; it does not establish that the network recognizes startup or that the low estimate is calibrated. A single image and its Q outputs cannot identify why the learned value differs so much, or distinguish waiting from active play.

The panel represents early random-play images, not all later-game situations. Some similar-looking scenes choose NOOP, and the single-frame input hides motion. State-dependent outputs alone cannot establish collision avoidance, effective navigation, or optimal play. The full RGB thumbnails provide human context; the CNN sees only the specified preprocessed crop.

4.5 MsPacman: edge cases and validation

Life loss is not automatically terminal: 6,228 life-loss transitions continue the game across 3,114 training games, while genuine game-over masks continuation. Zero rewards occur in 91.199% of decisions, with a longest within-game streak of 535 decisions; treating these intervals as stalls would change the task. Artificial frame-limit truncation instead retains the Bellman continuation, with coverage from separate reduced-cap tests because the main run has no truncations (Table 7). Finite Q outputs also need scrutiny: the short every-update target intervention increases the largest sampled magnitude from 24.58 to 91.88 without a NaN/Inf failure (Section 5.2). These cases test reward accounting, episode semantics, credit assignment, and learned value behavior, beyond input-format checks. Bellman diagnostics in Section 5.3 and recovery checks in Section 5.4 provide complementary correctness evidence.

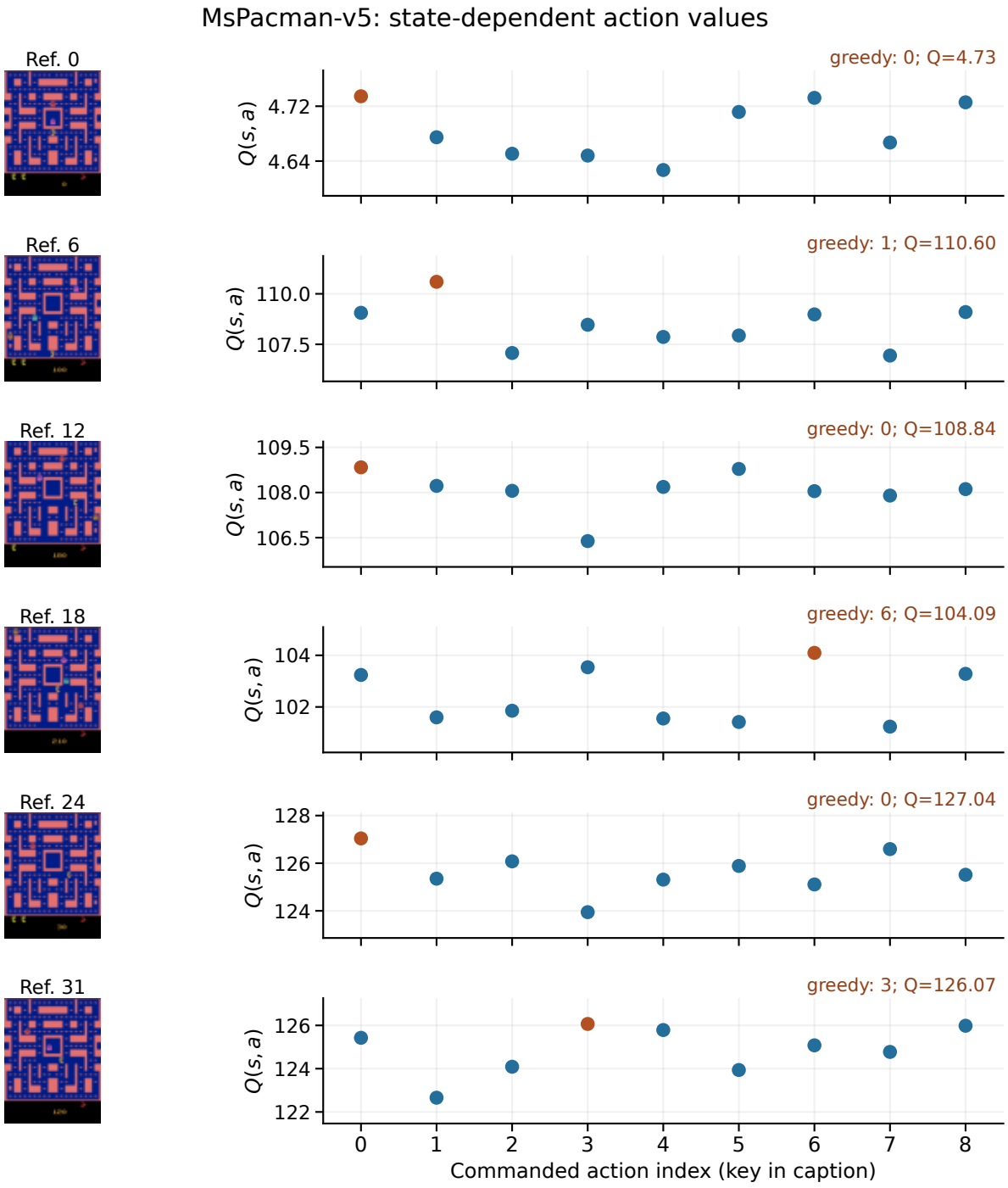


Figure 8: MsPacman image-conditioned Q preferences for references 0, 6, 12, 18, 24, and 31, evaluated by the selected episode-3,114 online CNN on the fixed preprocessed inputs. The horizontal axis is commanded action index; the vertical axis is predicted discounted raw-score return $Q(s, a)$, with a separate scale per row. Orange marks the greedy command; all nine values are plotted. Action indices: 0 NOOP, 1 UP, 2 RIGHT, 3 LEFT, 4 DOWN, 5 UPRIGHT, 6 UPLEFT, 7 DOWNRIGHT, 8 DOWNLEFT. Full RGB is shown only for context; reference 0 is the initial reset observation.

5 Scientific validation and edge cases

The experiments below test hypotheses about learning and numerical behavior, distinct from unit tests. Both games retain the same environment, architecture, optimizer, replay capacity, warmup, batch size, validation panel, and checkpoint selection rule within a controlled comparison, except for the explicitly changed factor. Actual exposure and evaluation sample size are reported: equal nominal caps do not guarantee identical decisions when stopping occurs at episode ends. Poor learning with finite computations is an experimental outcome, not by itself an implementation bug.

5.1 CartPole controlled experiments

Table 4 reports all eight completed runs. Ablations compare against *seed 11*, preserving the training seed, rather than against the stronger selected seed 33. Every selected checkpoint received 500 greedy resets on the common evaluation panel. Limits were 1,000 episodes or 80,000 decisions. The every-update and gamma-0.99 conditions reached the episode ceiling first, so their lower exposure is a material limitation of the comparison. Loss and extreme-Q diagnostics below summarize the entire training run, whereas evaluation returns describe validation-selected snapshots and final-state Q statistics describe the last network.

Table 4: CartPole controlled experiments and independent training seeds. Return mean and SD refer to each validation-selected model on 500 resets. "Perfect" is the percentage reaching 500 steps. Exposure is actual, not the nominal cap.

Condition (training seed)	Episodes	Decisions	Mean	SD_0	Perfect
Hard-500 baseline (11)	894	80,216	407.824	115.90	53.6%
Same configuration (22)	881	80,092	392.584	125.06	52.2%
Same configuration (33)	942	80,014	499.266	11.73	99.6%
Soft, old weight .999 (11)	948	80,132	223.292	62.36	0%
Hard every update (11)	1,000	15,769	9.36	0.77	0%
Recent-only batches (11)	794	80,183	385.14	138.63	57.8%
$\gamma = 0.90$ (11)	984	80,156	393.332	48.51	3.4%
$\gamma = 0.99$ (11)	1,000	58,648	163.25	58.88	0%

Training-seed sensitivity. The hypothesis is that initialization, exploration, reset, and replay randomness can materially alter learning under identical hyperparameters. Only the seed changes across 11, 22, and 33. The three evaluation means average 433.2; population SD *between run means* is 47.1 (sample SD 57.7), and the mean perfect-episode fraction is 68.47%. No run achieved 500/500 perfect episodes. Thus the best run is strong but not representative of reliably learned 500-step control across these three seeds.

Target lag: hard, soft, and every-update copies. The hypothesis is that lag limits moving-target feedback. At seed 11, the hard-500 baseline is compared with a soft update after every optimizer step,

$$\theta^- \leftarrow 0.999\theta^- + 0.001\theta, \tag{7}$$

and with hard copies every update. The coefficient 0.999 weights the *old* target, following the lecture convention [2]. Soft and every-update mean returns are lower than hard-500 by 184.532 and 398.464,

respectively. The every-update run's largest sampled absolute Q-value is approximately 2.23×10^7, versus 23.26 for hard-500; mean Huber losses are approximately 4.17×10^5 and 0.0754. This is severe *finite* value inflation relative to the unit-reward scale 20, accompanied by nearly immediate failures. Gradient clipping and nonfinite guards did not prevent it. These observations are consistent with harmful moving-target feedback in this run, not a proof of its sole cause. The different actual exposure and one training seed limit causal generalization. Soft-.999 also has a different effective lag from hard-500, so this is not an isolated test of smoothness.

Correlated replay. The hypothesis is that highly overlapping batches reduce sample diversity and may impair stability. Only sampling changes: the most recent 128 transitions in chronological order replace random minibatches, with capacity and all other settings fixed at seed 11. This retains memory and is not a no-replay method. Mean return falls by 22.684, but the perfect fraction rises from 53.6% to 57.8%. The recent condition has larger return SD and a lower loss 95th percentile (0.134 versus 0.239). The reward and success measures do not rank the schemes consistently, and lower regression loss does not imply better control.

Discount sensitivity. Changing only gamma to 0.90 or 0.99 tests the joint effect of planning horizon and value scale. Equation (6) gives infinite unit-reward references 10, 20, and 100 at gamma 0.90, 0.95, and 0.99. The measured final initial-state Q statistics are 10.05, 20.65, and 79.40: the first two approach their references, whereas the last remains about 20.6% below 100. These are network outputs at different final reset states, not exact policy values or theoretical bounds on the approximator.

Selected-policy mean returns are 393.332, 407.824, and 163.25, respectively. Matching the value scale therefore does not establish a strong policy. At 0.99, the run reaches the 1,000-episode ceiling after 58,648 decisions, compared with 80,216 at 0.95, and mean loss is 0.490 versus 0.0754. Its separate final-snapshot diagnostic mean is 102.22, also illustrating that the final Q statistic and validation-selected reward describe different networks. Finite experience, approximation error, and slower propagation through lagged targets are plausible contributors to the gap, but these data do not isolate their effects. Genuine failures shorten the reward stream; the 500-step time limit alone would give $100(1 - 0.99^{500}) \approx 99.34$, and this implementation bootstraps at truncation. Thus simply invoking that cutoff does not explain 79.40. These are unequal-exposure, finite-budget comparisons rather than an asymptotic ranking; the required primary gamma remains 0.95.

Altered initial states. The hypothesis is that high standard-reset performance need not imply recovery from larger perturbations. The selected seed-33 model is frozen. A 9×9 grid varies angle in $[-0.18, 0.18]$ rad and angular velocity in $[-1.5, 1.5]$ rad/s, with five nuisance cart resets per grid point (seeds 70,000–70,004). After reset, `env.unwrapped.state` is assigned the specified angle and angular velocity; forces, gravity, and time step are unchanged. All 405 episodes completed: mean 402.47, population SD 170.27, and 293/405 perfect episodes (72.35%). The minimum return is 1. This is a distribution-shift stress test, not standard CartPole evaluation. Some extreme states may be physically unrecoverable, and five nuisance resets per cell provide limited precision.

5.2 MsPacman learning, ablations, and short-budget replications

The main learning-evidence comparison holds the ten validation reset seeds fixed and changes training exposure. The selected-minus-untrained mean difference is 1,177 points. Its saved paired-bootstrap

interval is approximately 986–1,357 points and uses the panel that selected the model; it is selection-biased and exploratory, not a confirmatory improvement interval. The random-policy mean 252 and the intermediate checkpoint mean 707 provide additional context for the final validation mean 1,387.

The six ablations in Table 5 concern *early learning*, at about 5% of the main run's decision exposure. Baseline and slower-decay epsilon remain near 0.82 and 0.91, far above the shared floor 0.1. Every-update targets and slower decay both select their initial, untrained snapshots; each scores exactly 210 on all 30 diagnostic resets. Their score is a consequence of the stated selection rule at this budget, not the final performance of a mature trained agent. These comparisons cannot rank target strategies or exploration schedules after full annealing and longer training.

Table 5: MsPacman short-budget experiments, each capped at 100,000 decisions and evaluated on 30 diagnostic resets (700,000–700,029). All have gamma 0.99; each model is independently selected using ten validation resets. These are not full-budget replications of the 2,000,528-decision main run.

Changed factor (seed)	Games	Decisions	Mean	SD_0	Final ϵ
Baseline (41)	199	100,031	379	146.70	.8199
Training seed (52)	203	100,235	591	267.46	.8196
Training seed (63)	197	100,343	335	175.76	.8194
Target copy every update (41)	198	100,206	210	0	.8196
Recent-only batches (41)	197	100,299	394.333	203.86	.8195
Epsilon decay 1M (41)	197	100,341	210	0	.9097

Target-network hypothesis. Replacing hard-5,000 synchronization with every-update copies tests whether removing lag impairs learning, with all other factors fixed at seed 41 and matched decision caps. Diagnostic mean falls from 379 to 210. The largest sampled absolute Q rises from 24.58 to 91.88, and loss 95th percentile from 1.182 to 1.492. This finite inflation is much smaller than the CartPole ablation's pathology. The training Q/loss summaries describe updated networks, whereas the diagnostic return describes the selected initial snapshot. They support a configuration-specific early-learning finding, not a general failure of every-update targets.

Replay hypothesis. Recent chronological batches of 32 replace uniform batches, keeping capacity, warmup, update ratio, seed, and all other settings fixed. Mean score changes by +15.333, while loss 95th percentile increases to 1.763. The saved paired 95% bootstrap interval for the score difference is $[-73.68, 124.67]$, spanning zero. This is inconclusive, rather than evidence of equivalence or of replay diversity being unimportant later.

Exploration hypothesis. Doubling the linear decay horizon from 500,000 to 1,000,000 decisions tests whether prolonged exploration changes early learning. Both endpoints remain unchanged. Final epsilons are 0.8199 and 0.9097; diagnostic means are 379 and 210. This measures the effect of maintaining more random actions during early learning; neither schedule has completed annealing.

Uncertainty and replication scope. The saved paired intervals use common diagnostic reset seeds, 10,000 percentile bootstrap resamples, and variant-minus-baseline scores. Every-update and slow-decay differences are both −169, with intervals $[-221.33, -117.33]$ and $[-222, -116.66]$.

These intervals are conditional on the selected models, describe reset uncertainty, have no multiple-comparison adjustment, and do not measure uncertainty across training seeds. Independent short seeds 41, 52, and 63 have mean-of-means 435 and between-run population SD 111.8. Only one seed has full-budget training, so full-budget reliability remains unmeasured.

5.3 Trajectory and Bellman diagnostics

For CartPole, the frozen selected model and reset seed 31,415 produce a complete ten-step trace of states, both Q-values, actions, rewards, next states, end flags, and checkpoint-based targets. All 10/10 actions match online argmax. An independent float64 NumPy implementation of the saved target network's linear/ReLU layers recomputes Bellman targets for 40 real sampled transitions, including four true terminals and four truncations. Maximum absolute error is approximately 3.02×10^{-6}. The analytic example in Section 2.1 separately checks the semantic difference between the flags.

Table 6: All ten CartPole diagnostic decisions, reset seed 31,415. States are measured before each action; Q-values use the frozen selected online network. y uses its saved target network on the next state, not historical collection-time weights. Position is in m, velocity in m/s, angle in rad, angular velocity in rad/s. L/R denote actions 0/1; every row has reward 1 and both end flags false.

t	x	$\dot{x}$	θ	$\dot{\theta}$	Q_L	Q_R	a	r	y
1	−0.000	−0.038	0.036	0.034	20.630	20.672	R	1	20.584
2	−0.001	0.156	0.037	−0.247	20.603	20.635	R	1	20.582
3	0.002	0.351	0.032	−0.528	20.560	20.618	R	1	20.615
4	0.009	0.545	0.021	−0.811	20.561	20.624	R	1	20.616
5	0.020	0.740	0.005	−1.096	20.549	20.594	R	1	20.518
6	0.035	0.935	−0.017	−1.388	20.457	20.466	R	1	20.409
7	0.054	1.131	−0.045	−1.685	20.331	7.346	L	1	20.480
8	0.076	0.936	−0.078	−1.407	20.418	7.527	L	1	20.459
9	0.095	0.742	−0.106	−1.140	20.418	8.446	L	1	20.389
10	0.110	0.548	−0.129	−0.882	20.358	11.719	L	1	20.439

Initially both action estimates are near 20.6. By step 7, angular velocity is −1.685 rad/s and Q_R falls to 7.346 while Q_L remains 20.331, switching the greedy command to left. Since the weights are frozen, this is a state-dependent change, not a learning update. The trace makes the relationship among observation, action preference, and target explicit; its ten steps do not establish long-run stability or representative state occupancy.

For MsPacman, seed 314,159 supplies ten initial decisions and ten later decisions (101–110), with all nine Q-values and exact processed state/next-state image arrays. All 20/20 actions match argmax. The later window supplements potentially repetitive startup frames. On 36 real replay transitions, including five terminals and no truncations, independent float64 *scalar arithmetic* on saved target Q-values has maximum discrepancy approximately 8.54×10^{-6}. This is not an independently implemented CNN. Synthetic time-limit cases provide the truncation coverage absent from these real transitions. Both games' diagnostics use saved-checkpoint weights; neither reconstructs historical targets at transition collection time.

5.4 Engineering verification and reproducibility

Checkpoint inference gives zero same-CUDA CartPole reload discrepancy and maximum CPU–CUDA differences approximately 5.72×10^{-6} over 128 CartPole probes and 0.00899 over 20 MsPacman images, with greedy actions agreeing in both cases. These finite checks support inference consistency across the tested devices.

Repeating MsPacman seed 41 under the same GPU/software matched the main run's first 100,031 recorded transitions, including scalar fields and image checksums. Online and target parameters matched exactly at episode 100, the last common *saved* checkpoint; no claim is made about unsaved final-prefix weights. Checksums can collide, so matching checksums alone are not proof of pixel identity. This is a repeated prefix, not a second complete main experiment.

Full Atari recovery retains networks, optimizer, replay, counters, RNGs, and reference/selection metadata at complete-game boundaries; model-only snapshots cannot resume the full process. A four-game smoke compared uninterrupted execution, a two-plus-two resume, and rollback from game two. With target interval 50 *for this test only*, it completed 471 updates and nine target copies. Both maximum online-parameter discrepancies were zero, and restored logs contained unique game indices 1–4 after archival rollback. This supports tested state and log restoration, not convergence; abrupt loss can still discard work since the latest boundary. The technical source README documents the required sidecars.

The focused suites contain 18 passing CartPole tests and 12 passing MsPacman tests. They check replay underflow/wraparound, chosen-action indexing, epsilon extremes and global-step schedules, known Huber values, frozen targets and exact synchronization, and evaluation isolation. Atari also tests alias equivalence, headless execution, signed image arithmetic, and replay RAM guards. Together with source hashes, these checks support specific implementation properties; they do not substitute for the learning interventions or rollout evidence.

5.5 Environment-specific edge-case evidence

Table 7 groups task-specific failure modes by environment. It distinguishes observed policy behavior from analytic or reduced-cap checks.

Atari image checksums were unchanged on 2.41% of main decisions; this is an instrumented checksum statistic rather than proof of identical images. Real main/evaluation games did not hit the native frame limit, so truncation evidence comes from explicitly labeled reduced-cap tests.

Table 7: Reinforcement learning edge cases and their evidence. Analytic controllers and shortened frame caps check episode/target semantics and do not replace learned-policy evaluation. General engineering checks are summarized in Section 5.4.

Potential failure	Handling	Validation evidence
CartPole		
Early failure or omitted final reward	End on either flag; count every reward	One-step failure returns 1; analytic controller reaches the 500-step limit; selected DQN reaches it in 498/500 resets
Time cutoff treated as absorbing	Only termination masks continuation	Known targets 1 for failure versus 4.8 for truncation; four real truncated transitions in independent Bellman checks
Standard-reset success mistaken for recovery	Freeze the policy and perturb initial angle/angular velocity	293/405 perfect stress episodes (72.35%); mean 402.47 and SD 170.27 under unchanged dynamics
MsPacman		
Life loss mistaken for game-over	Continue through nonterminal life loss	6,228 nonterminal life losses and 3,114 genuine game-overs; scores include every life
Sparse rewards mistaken for a stall	Keep stepping and retain zero rewards	91.199% zero-reward decisions; longest within-game interval 535 decisions; raw rewards span 0–1,610
Frame limit masks valid continuation	Bootstrap at truncation; mask genuine game-over	Zero main/test truncations; separate native 12-frame and wrapper nine-decision tests cover time cutoffs
Finite Q inflation escapes NaN/Inf guards	Inspect value/loss trajectories as well as finiteness	Short every-update run reaches sampled $\|Q\| = 91.88$ versus 24.58 for baseline; nonfinite injection and black/white probes check guards
Image arithmetic changes the observation	Int32 channel sum, sum-448 mask, int8 CHW	Known black/white pixel values, signed range, shape, and layout checks validate the specified transform

6 Discussion and further experiments

The CartPole best run approaches the 500-step goal on its tested reset distribution, but its two failures, seed sensitivity, right-biased policy slice, and lower stress-test success qualify any robustness claim. The target ablation provides unusually clear evidence that finite value estimates can become scientifically implausible while software guards continue to pass. The replay ablation is less decisive: return mean and perfect-score rate rank variants differently. These results reinforce the lecture's caution that loss reduction and occasional high returns are insufficient measures of successful control.

MsPacman exhibits improved raw scores relative to initialization, random actions, and early checkpoints, together with state-dependent action preferences. Its computational cost is greater, but no CPU/GPU speed comparison was measured. Single-frame observations hide velocity and temporal context; the assignment's crop and intensity mask discard information. Sticky actions, sparse rewards, bootstrap approximation, and a reused small validation panel complicate inference. The decision-limited run ends below the suggested game count, only one full-budget seed is available, and short ablations remain highly exploratory. Neither rising Q-values nor the best validation score proves calibrated values, global optimality, or mathematical convergence.

6.1 Selection and evidence limitations

Within each run, checkpoints were selected by validation rather than the final test. CartPole's final primary designation across three seeds also reads only validation scores. However, per-run test summaries had already been inspected before that final designation. Reporting every seed and avoiding subsequent hyperparameter tuning reduces selective presentation, but the selected best-run test should not be described as a fully preregistered, untouched across-seed confirmatory study. MsPacman's final 500-game panel is separate from checkpoint selection and short-run diagnostics. In both games, repeated use of ten validation seeds can favor a noisy maximum. Additional fresh panels and more independent training runs would strengthen generalization claims.

6.2 What would be tested with an additional three months?

Month 1: reliability and replication. Preregister matched decision/frame budgets, model selection, and a final untouched panel. Prioritize 10 full-budget seeds per game, executed sequentially within a declared compute allocation, before expanding the hyperparameter search. Continue CartPole beyond its late improvement and extend MsPacman beyond its present cap; evaluate both at common exposure points. Report distributions of seed-level means, CartPole success probability, score quantiles, sample efficiency, wall time, and between-run uncertainty. Sweep learning rate, exploration horizon, and target lag one factor at a time. Match effective lag when comparing hard and soft updates, and do not let an episode ceiling give weak policies much less interaction.

Month 2: controlled algorithm extensions. Keep Classic DQN as the reference and separately test Double DQN [8], dueling heads [9], prioritized replay [6], and multi-step returns. Ask whether they improve reward per decision, reduce finite value inflation, or alter seed variability under equal experience and comparable update budgets. Compare Q estimates with independently estimated discounted rollout returns at fixed states to assess calibration, not just Q magnitude. Factorial follow-ups would test interactions only after the individual interventions are understood.

Month 3: generalization, representation, and robustness. Compare single-frame and four-frame inputs while accounting for increased replay memory. Stratify Atari diagnostics by later levels, life loss, sparse-reward intervals, and ghost encounters. Expand CartPole's recovery map across all four state dimensions and compare failures with physically recoverable regions. Test observation noise, delay, and explicitly labeled dynamics shifts; report return quantiles, recovery rates, and action consistency. Replicate recovery across devices and software versions and add a small number of additional Atari benchmarks only after within-game reliability is measured. These are proposed studies, not completed results of this assignment.

7 Reproducibility and source availability

GitHub username: `yuxuanliang1`. **Repository name:** `MLDS490-Assignment2`.
Public source link: https://github.com/yuxuanliang1/MLDS490-Assignment2.

7.1 Execution environment and retained evidence

All substantive neural-network training ran sequentially on one NVIDIA GeForce RTX 5060 Ti (nominal 16 GB), compute capability 12.0, on `mlds-deepdish5.ads.northwestern.edu`. The recorded build supports `sm_120`. Inputs, networks, targets, losses, gradients, and Adam moment tensors use CUDA; environment stepping and replay storage use CPU. The saved environment is Python 3.12.6, PyTorch 2.14.0 with CUDA 13.0, driver 615.71.09, Gymnasium 1.4.0, ALE 0.10.2, NumPy 2.5.3, Matplotlib 3.11.2, pandas 3.0.5, pytest 9.2.0, and psutil 7.2.2. Seeds cover Python, NumPy, PyTorch, CUDA, replay, behavior, action spaces, and resets; deterministic kernels are requested. Exact repetition across versions or hardware is not guaranteed.

The public repository includes implementation and test code, the compiled report, selected original CSV/JSON measurements, PNG figures, and the fixed Atari image panel. These support independent verification of both 500-episode statistics, training curves, and reported comparisons. Selected model weights, full transition and optimizer logs, replay recovery archives, course PDFs, and editable LaTeX sources remain local. Therefore exact selected-policy inference requires obtaining the unpublished checkpoints; rerunning training can select different policies. CartPole snapshots support inference, while full MsPacman recovery additionally requires its optimizer/replay/RNG state and companion files. The repository's `assignment2/PUBLIC_RESULTS.md` specifies the available evidence.

7.2 Environment setup and reproduction commands

Run the following Bash setup from the repository root on Linux with Python 3.12. The root `requirements.txt` includes `assignment2/requirements_mspacman.txt`, which in turn includes the shared `assignment2/requirements.txt`. These pin the recorded package versions.

```
python3.12 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m assignment2.verify_public_results
```

The torch version pin does not select a CUDA wheel flavor or guarantee the recorded GPU build. For Blackwell, install a matching torch 2.14.0 build with `sm_120` support using the official PyTorch installation instructions, then verify `torch.version.cuda` and `torch.cuda.get_arch_list()` before training. The root README links the installer and gives hardware checks. For Atari, `ale-py` and the MsPacman ROM must be available. The recorded ALE installation supplied the ROM; no separate AutoROM step is assumed. The project's `make_env` registers and verifies the literal `MsPacman-v5` alias against `ALE/MsPacman-v5`. This CPU smoke checks ROM discovery, both environments, and one transition without training:

```
CUDA_VISIBLE_DEVICES=-1 python - <<'PY'
import gymnasium as gym
from ale_py import roms
from assignment2.mspacman.environment import make_env
assert roms.get_rom_path("ms_pacman").is_file()
for env in (gym.make("CartPole-v1"), make_env()):
    try:
        obs, _ = env.reset(seed=0)
        env.step(0)
        print(env.spec.id, obs.shape, env.action_space.n)
    finally:
        env.close()
```

```
PY
CUDA_VISIBLE_DEVICES=-1 python -m pytest assignment2/tests -q
```

Training and evaluation CLIs enforce deepdish5 and exactly one usable NVIDIA GPU. Inspect the current inventory, set `GPU_UUID` to an idle device, and export visibility before importing torch. Use fresh output roots. The limits below reproduce the measured budgets rather than larger generic defaults; `finalize` reads the run's selection JSON, evaluates exactly 500 greedy episodes, and plots the results.

```
nvidia-smi --query-gpu=uuid,name,memory.used,utilization.gpu --format=csv
# Set GPU_UUID to one currently idle UUID from the inventory.
export CUDA_VISIBLE_DEVICES="${GPU_UUID:?Set GPU_UUID first}"
export CUBLAS_WORKSPACE_CONFIG=:4096:8
export OMP_NUM_THREADS=1
export MPLBACKEND=Agg
CP=assignment2/results/reproduce_cartpole
MS=assignment2/results/reproduce_mspacman/main_seed41
python -m assignment2.cartpole.train --output "$CP/seed33" \
  --seed 33 --episodes 1000 --max-steps 80000
python -m assignment2.cartpole.validate finalize --run "$CP/seed33"
python -m assignment2.mspacman.train --output "$MS" --seed 41 \
  --episodes 4000 --max-steps 2000000 --max-hours 4
python -m assignment2.mspacman.validate finalize --run "$MS"
```

Typical primary training times were about 3 minutes for CartPole and 54.48 minutes for MsPacman; Atari evaluation took 4.19 minutes. These machine-specific times exclude installation and the wider validation suite. Native episode-boundary stopping allows the documented decision-cap overshoots.

The full CartPole suite is an *alternative* to the single-run route. A different root avoids a seed-33 directory collision. It first establishes the matched seed-11 baseline, then trains the seven other conditions sequentially. The last four commands below provide algorithm diagnostics and the six short Atari comparisons described in Section 5.

```
CPS=assignment2/results/reproduce_cartpole_suite
python -m assignment2.cartpole.train --output "$CPS/main_seed11" \
  --seed 11 --episodes 1000 --max-steps 80000
python -m assignment2.cartpole.validate finalize --run "$CPS/main_seed11"
python -m assignment2.cartpole.validate suite --root "$CPS" --max-steps 80000
python -m assignment2.cartpole.validate primary --root "$CPS"
python -m assignment2.cartpole.validate diagnostics --run "$CPS/seed33"
python -m assignment2.cartpole.validate robustness --run "$CPS/seed33"
python -m assignment2.mspacman.validate short-suite \
  --root assignment2/results/reproduce_atari_validation --steps 100000
python -m assignment2.mspacman.validate diagnostics --run "$MS"
```

The source README provides individual evaluation/plot commands, recovery and rollback tests, and the repeated-prefix protocol. Evaluation outputs must be fresh: do not run `finalize` and standalone evaluation into the same directory. Full plotting requires checkpoints; verification of published measurements does not. The `-preferences-only` plotting option uses the saved Atari images and Q-values, with a copy of the evidence directory as the output workspace. Compiling the LaTeX requires only the supplied sections, PDF graphics, and bibliography, with no experiment execution or shell escape.

References

[1] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013. https://arxiv.org/abs/1207.4708.

[2] Diego Klabjan. Deep Q-learning: DQN enhancements. MLDS 490 course lecture slides, Northwestern University. Supplied as Week 6 DQN.pdf, especially pp. 6–15 and 18–20.

[3] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018. https://arxiv.org/abs/1709.06009.

[4] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, Stig Petersen, Charles Beattie, Amir Sadik, Ioannis Antonoglou, Helen King, Dharshan Kumaran, Daan Wierstra, Shane Legg, and Demis Hassabis. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015. https://doi.org/10.1038/nature14236.

[5] Siqiao Mu. MLDS 490: Lab week 6. Northwestern University course lab handout, November 9, 2026. Supplied as MLDS_Lab_Week_6_1.pdf, especially pp. 4–10.

[6] Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016. https://arxiv.org/abs/1511.05952.

[7] Mark Towers, Ariel Kwiatkowski, Jordan Terry, et al. Gymnasium: A standard interface for reinforcement learning environments. arXiv:2407.17032, 2024. https://arxiv.org/abs/2407.17032.

[8] Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2016. https://arxiv.org/abs/1509.06461.

[9] Ziyu Wang, Tom Schaul, Matteo Hessel, Hado van Hasselt, Marc Lanctot, and Nando de Freitas. Dueling network architectures for deep reinforcement learning. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1995–2003, 2016. https://proceedings.mlr.press/v48/wangf16.html.